SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS: **Total Marks: 30**

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly. **Questions:**

1. Identify appropriate **HTML5 semantic elements** missing in the structure.
2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.
5. Suggest improvements for **usability and accessibility**.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
  <head>
    <title>Company</title>
  </head>
  <body>
    <div>
      <h1>Welcome
      <p>About us
    </div>
  </body> </html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

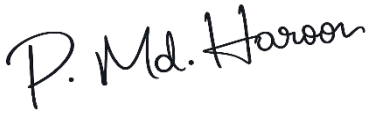
Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

Code of Conduct:

I **Mohamed Haroon P (192511313)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, wellexplained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

QUESTION 1 — Online Symposium Portal

Scenario: *A college symposium registration form submits via POST to /register. The given markup is a bare fragment (no DOCTYPE/head/body wrapper, no labels).*

Given Code

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

Q1 — Identify Appropriate HTML5 Semantic Elements Missing

The snippet is an isolated fragment, not a complete document, and lacks the structural and labeling elements a standards-compliant registration page needs:

Missing Element	Why It's Needed
<!DOCTYPE html>	Forces the browser into Standards Mode rather than legacy Quirks Mode — without it, layout/box-model rendering becomes unpredictable.
<html lang="en">, <head>, <body>	The fragment has no document skeleton at all — a <form> cannot legally exist outside a <body> in a real page.
<meta charset="UTF-8">	Declares character encoding; omitting it risks misrendering special characters (e.g., accented names).
<title>	Every document needs a descriptive title for the browser tab, bookmarks, and screen readers.
<header> / <main>	Semantically wraps the page banner and the primary registration content, improving document outline and SEO.
<label for="...">	Each <input> has no associated label — a serious accessibility gap; screen readers cannot announce what each field is for.
<fieldset> / <legend>	Groups related registration fields under a named legend, which assistive technology announces as a group.

Q2 — Modify the Structure to Make It Standards-Compliant

Corrected, Standards-Compliant HTML5

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Symposium Registration</title>
</head>
<body>
  <header>
    <h1>College Symposium 2026 — Registration</h1>
  </header>

  <main>
    <section>
      <form action="/register" method="POST">
        <fieldset>
          <legend>Participant Details</legend>

          <label for="name">Full Name:</label>
          <input type="text" id="name" name="name" required autocomplete="name">

          <label for="email">Email Address:</label>
          <input type="email" id="email" name="email" required autocomplete="email">

          <button type="submit">Submit</button>
        </fieldset>
      </form>
    </section>
  </main>

  <footer>
```

```
    <p>&copy; 2026 College Symposium Committee</p>
  </footer>
</body>
</html>
```

Q3 — Interpret the HTTP Request Generated During Form Submission

Because method="POST" is specified, the browser packages the form field values into the HTTP request BODY (not the URL) using the default enctype, application/x-www-form-urlencoded:

Raw HTTP Request (as sent by the browser)

```
POST /register HTTP/1.1
Host: symposium.college.edu
Content-Type: application/x-www-form-urlencoded
Content-Length: 34
Connection: keep-alive

name=Harsh+Kumar&email=harsh%40college.edu
```

- Request line: POST /register HTTP/1.1 — method, target resource path, and HTTP protocol version.
- Content-Type header tells the server how to decode the body (URL-encoded key=value pairs, with spaces as '+' and special characters percent-encoded, e.g. @ becomes %40).
- Content-Length tells the server exactly how many bytes of body to read — critical for POST, since (unlike GET) the payload lives in the body, not the request line.

Q4 — Analyze the HTTP Response Cycle from Server to Browser

Step	What Happens
1	Server receives the POST request and parses the URL-encoded body to extract name and email.
2	Server validates and processes the data (e.g., inserts a new registration record into its database).
3	Server constructs an HTTP response — ideally a 303 See Other redirect to a confirmation page, NOT a direct 200 OK with HTML (this is the Post/Redirect/Get pattern, explained below).
4	Browser receives the response, reads the Location header, and automatically issues a fresh GET request to that URL.
5	Server responds to that GET with 200 OK and the confirmation page's HTML, which the browser then renders.

Example Response to the Original POST (Post/Redirect/Get Pattern)

```
HTTP/1.1 303 See Other
Location: /register/confirmation
Content-Length: 0
```

Why redirect instead of responding directly? If the server instead sent 200 OK with the confirmation HTML directly in response to the POST, the confirmation page itself would be tied to that POST request in browser history. Refreshing the page (or pressing Back then Forward) would trigger the browser's "Confirm Form Resubmission" warning and could re-submit the registration a second time. The Post/Redirect/Get (PRG) pattern avoids this entirely — the final page in history is a GET, which is always safe to refresh.

Q5 — Suggest Improvements for Usability and Accessibility

- Associate every input with a <label for="..."> (shown in Q2) so screen readers announce field purpose, and clicking the label focuses the field.
- Add required and appropriate autocomplete attributes (name, email) so browsers can offer autofill, reducing user effort.
- Use aria-describedby to link inputs to inline validation/error messages, and aria-live="polite" regions so screen-reader users are notified of validation errors without losing their place.
- Ensure sufficient color contrast and visible :focus outlines for keyboard-only navigation — never remove focus outlines purely for aesthetics.
- Provide clear client-side validation feedback (e.g., "Please enter a valid email") in addition to the required attribute, always paired with server-side validation (client-side alone is not secure or reliable).

- Make the submit button's label descriptive (e.g., "Register for Symposium" rather than a bare "Submit") so its purpose is clear out of context — helpful for screen-reader users navigating by button list.

✓ Key Takeaway: The fragment is missing its entire document skeleton, semantic landmarks (header/main/footer), and label associations. The corrected version wraps the form in valid HTML5 with proper labels and a fieldset; the POST request carries form data URL-encoded in the body (not the URL); and the server should respond with a 303 redirect (Post/Redirect/Get) rather than a direct 200 OK, to prevent duplicate-submission issues on refresh.

QUESTION 2 — Cross-Browser Compatibility Issue

Scenario: *A company website renders inconsistently across browsers. No DOCTYPE is declared, and the <h1> and elements are never explicitly closed.*

Given Code

```
<html>
  <head>
    <title>Company</title>
  </head>
  <body>
    <div>
      <h1>Welcome
      <p>About us
    </div>
  </body>
</html>
```

Q1 — Identify Improper Nesting and Missing Closing Tags

Issue	Detail
Missing <!DOCTYPE html>	No doctype declaration at all — the single most consequential omission here (see Q2).
<h1>Welcome not closed	No </h1> anywhere in the markup.
** About us not closed**	No anywhere in the markup.
Implicit tag closure relied upon	Only one explicit closing tag, , appears for three opened elements (, <h1>,) — the browser's error-recovery parser must infer where h1 and p actually end.

Q2 — Analyze How Different Browsers May Interpret This Structure

The missing DOCTYPE is the critical issue: without <!DOCTYPE html>, older or non-evergreen browsers can fall back into Quirks Mode instead of Standards Mode. In Quirks Mode, box-model calculations (how width/height interact with padding and border), default font sizing, and table-cell sizing are deliberately rendered the way 1990s browsers did — which differs from modern standards-based rendering and differs between browser engines, producing exactly the "inconsistent layout" symptom described.

Separately, every modern browser implements the WHATWG HTML parsing algorithm, which defines precise error-recovery rules for malformed markup like unclosed <h1>/<p> tags — so evergreen browsers (current Chrome, Firefox, Safari, Edge) will actually construct the same DOM tree from this fragment, auto-closing <h1> and <p> at predictable points. The real cross-browser risk here is less about today's major browsers disagreeing on error recovery, and much more about (a) Quirks Mode being triggered at all due to the missing DOCTYPE, and (b) older/legacy browser versions with less complete adherence to the modern parsing spec.

Q3 — Identify Missing Semantic Elements

- <!DOCTYPE html> — forces Standards Mode (addressed above).
- <meta charset="UTF-8"> — no character encoding declared.
- <meta name="viewport" content="width=device-width, initial-scale=1.0"> — without it, mobile browsers render at a desktop-width virtual viewport and scale down, which is itself a major source of "looks different across devices/browsers" complaints.
- <header> — should wrap the site branding/"Welcome" heading rather than a bare, unstyled .
- <section> — should wrap the "About us" content block as a distinct, labeled content region.
- <main> — should wrap the page's primary content, giving assistive technology and search engines a clear landmark.

Q4 — Corrected W3C-Compliant HTML Structure

Corrected HTML5

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Company</title>
</head>
<body>
  <header>
    <h1>Welcome</h1>
  </header>

  <main>
    <section>
      <p>About us</p>
    </section>
  </main>
</body>
</html>
```

Q5 — Techniques to Ensure Cross-Browser Compatibility

- Always declare <!DOCTYPE html> at the very top of every document to guarantee Standards Mode rendering across all browsers.
- Validate markup against the W3C Markup Validator (validator.w3.org) during development to catch unclosed tags and invalid nesting before they ever reach production.
- Use a CSS reset or normalize.css to neutralize each browser's own default stylesheet differences (default margins, font sizes, list styling, etc.) as a consistent baseline.
- Use well-supported semantic HTML5 elements (header, main, section, footer); for any genuinely legacy-browser target, pair them with a polyfill such as html5shiv so very old browsers (pre-HTML5 element support) still recognize them as block-level elements.

- Use feature detection (e.g., via Modernizr, or native CSS @supports) rather than browser-sniffing, so the page adapts to actual capability rather than guessing from a user-agent string that can be spoofed or simply wrong.
- Test across real target browsers and devices (Chrome, Firefox, Safari, Edge; desktop and mobile) using tools like BrowserStack, in addition to automated regression/visual-diff testing in CI.
- Follow progressive enhancement: build a functional baseline experience first, then layer on advanced CSS/JS features for browsers that support them, rather than requiring the latest features just to render correctly at all.

✓ Key Takeaway: The layout inconsistency traces mainly to the missing <!DOCTYPE html> (triggering inconsistent Quirks-Mode rendering) plus unclosed <h1>/<p> tags and missing semantic landmarks. The fix: add the doctype, explicitly close every tag, wrap content in header/main/section, add a viewport meta tag, and validate against W3C standards — paired with a CSS reset and real multi-browser testing going forward.

QUESTION 3 — Form Submission Failure

Scenario: *A login form fails to send data to the server. It uses method="GET" and its button has type="button" (not the default submit).*

Given Code

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

Q1 — Identify Issues in the Form Configuration

Issue	Why It's a Problem
button has type="button"	This is the root cause of the described failure (see Q2) — a type="button" element has NO default form-submission behaviour at all.
method="GET" on a login form	GET appends all field values — including the plaintext password — to the URL as a query string. This is a serious security flaw for credential data (see Q3).
No required attributes	Nothing stops the form from being submitted with empty username/password fields once submission is actually fixed.
No <label> elements	Same accessibility gap as Scenario 1 — fields have no programmatically associated text.

Q2 — Analyze Why the HTTP Request Is Not Triggered

In HTML, a <button> element's type attribute determines its behaviour inside a <form>, and the three valid values behave very differently:

button type	Default Behaviour
submit (the default if type is omitted)	Submits the enclosing form — triggers the HTTP request.
reset	Resets all form fields to their initial values — no HTTP request.
button (used here)	Does absolutely nothing on its own — a plain, inert clickable element with no built-in form interaction.

Because this button is explicitly type="button", clicking "Login" fires no submit event and triggers no navigation or HTTP request whatsoever — the browser simply does nothing, which precisely matches the reported symptom ("fails to send form data to the server"). The only way a type="button" element could submit the form is via explicit JavaScript (e.g., an onclick handler calling form.submit()), and no such script is present in the given markup.

Q3 — Interpret the Role of GET Method in This Scenario

Even setting aside the button issue, method="GET" is fundamentally the wrong choice here. If it were triggered, the browser would construct a request like this:

What a GET Submission Would Produce (if the button issue were fixed)

```
GET /submit?username=harsh&password=MySecret123 HTTP/1.1
Host: app.example.com
```

- GET is defined by the HTTP specification as a safe, idempotent method intended for retrieving a resource — it was never designed to carry sensitive, state-changing data.
- Because the data rides in the URL itself, the plaintext password ends up stored in the browser's address bar, browsing history, and bookmarks; it is also commonly recorded in plaintext in web-server access logs and any intermediate proxy logs along the request path.
- A URL containing credentials can also be accidentally shared (copy-pasted, screen-shotted) or cached by intermediary systems — none of which happens with POST, where the data travels in the request body instead of the URL.

Q4 — Propose a Corrected Implementation

Corrected HTML

```
<form action="/submit" method="POST">
  <label for="username">Username:</label>
  <input type="text" id="username" name="username" required autocomplete="username">

  <label for="password">Password:</label>
  <input type="password" id="password" name="password" required autocomplete="current-password">
```

```
  <button type="submit">Login</button>
</form>
```

Two changes fix both root problems at once: method is changed from GET to POST (so credentials travel in the request body, not the URL), and the button's type is changed from "button" to "submit" (restoring the browser's default form-submission behaviour, so clicking Login now actually triggers the HTTP request).

Q5 — Suggestions for Secure Data Transmission

- Always use method="POST" for any form carrying credentials or other sensitive/state-changing data — never GET.
- Serve the login page (and the /submit endpoint) exclusively over HTTPS, so the POST body is encrypted in transit via TLS — POST alone does not encrypt data, it only keeps it out of the URL; HTTPS is what actually protects it from network eavesdropping.
- Never store passwords in plaintext server-side — hash them with a strong, salted algorithm (e.g., bcrypt or Argon2) so even a database breach doesn't expose usable credentials.
- Include a hidden CSRF token field in the form and validate it server-side, to prevent cross-site request forgery attacks that trick a logged-in user's browser into submitting unintended requests.
- Use autocomplete="current-password" (not off) so password managers work correctly, improving both usability and the ability to use strong, unique passwords.
- Implement server-side rate-limiting / account lockout after repeated failed login attempts, to mitigate brute-force password guessing — a defense that client-side HTML alone cannot provide.
- After a successful login, issue the session identifier as a Secure, HttpOnly, SameSite cookie, so it cannot be read by client-side JavaScript (mitigating XSS-based session theft) and is only ever sent over HTTPS.

✓ Key Takeaway: The form never submits because its button has type="button" (no default submit behaviour) — and even if that were fixed, method="GET" would leak the plaintext password into the URL, browser history, and server logs. The corrected form uses method="POST" with type="submit", and must additionally be served over HTTPS with server-side password hashing, CSRF protection, and rate-limiting to be genuinely secure.